LAB REPORT:

BUILDING AI AGENTS WITH LANGGRAPH



langgraph

LangChain

→ 1 INTRODUCTION

This report summarizes the practical lab on building AI agents with LangGraph.
The lab moves from a one-node graph that echoes a message to a multi-agent university assistant with a classifier, specialized agents, tools, memory, conditional routing, and error handling.

The aims were to:

- distinguish a conventional LLM chain from an agentic workflow
- apply the core LangGraph concepts: state, nodes, edges, conditional edges and graph execution
- add tools and memory to an agent
- build a router that dispatches questions to specialized agents
- visualize, test and evaluate the resulting graphs.

All code was executed in a Jupyter notebook, and the results quoted below come from the recorded outputs of that run.

→ 2. ENVIRONMENT AND SETUP

PYTHON	3.12
LANGGRAPH	1.2.19
LANGCHAIN	1.4.8
LANGCHAIN-OPENAI	1.7.3
LANGCHAIN-COMMUNITY	0.4.2

LLM provider. The original lab used OpenAI's GPT-4o-mini through ChatOpenAI . Because of API usage limits, the implementation switched to OpenRouter (model openrouter/free , temperature=0) while keeping the same LangChain interface. Only the LLM configuration changed; the graphs, tools and routing logic are identical. The API key is read from an environment variable or entered through getpass , so it is never hard coded in the notebook.

→ 3.CONCEPTUAL QUESTIONS

#1

THE ROLE OF LANGCHAIN

LangChain is a framework of reusable components and integrations for LLM applications: a common interface to chat models, prompt templates, message types, tools, output parsers and retrievers. It makes it easier to switch providers without rewriting the application.

#2

THE ROLE OF LANGGRAPH

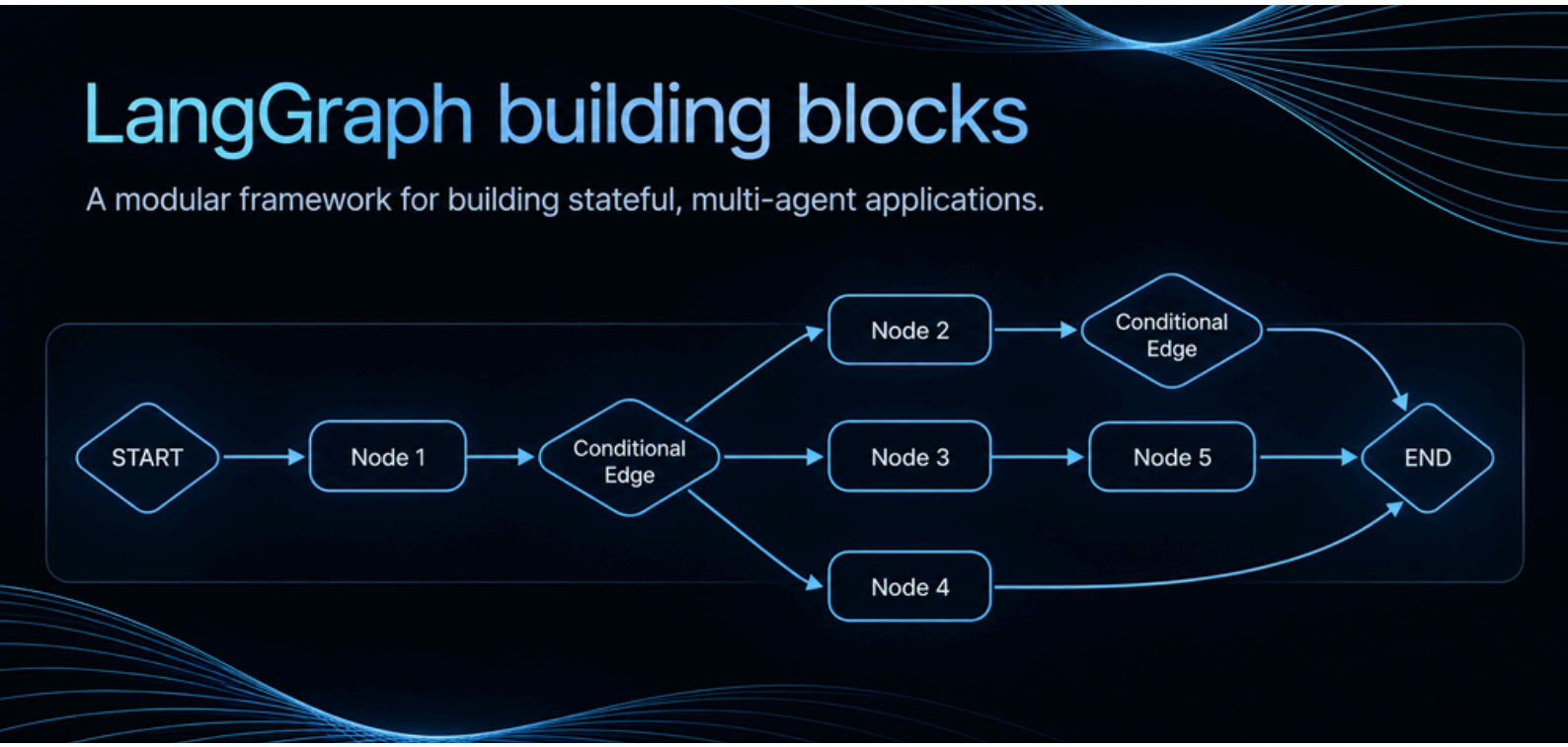
LangGraph is an orchestration framework for stateful, multi-step agent workflows. An application is a graph of nodes (tasks) and edges (what happens next), with support for conditional branching and loops. Unlike a linear chain, it lets an agent decide, call tools, evaluate results and repeat steps.

#3

LLM

An LLM is a model trained on large text corpora that generates output by predicting tokens. In an agent it provides language understanding and reasoning, and it can decide that a tool is needed. It does not execute external actions itself: the surrounding framework runs tools, manages state and handles external systems.

→ LANGGRAPH BUILDING BLOCKS



NODE

is a unit of work in LangGraph workflow. It is usually a Python function that receives the current state, performs a specific task, and returns a partial update to the state.

EDGE

is a direct connection between nodes that defines the execution flow of a graph. It determines which node should execute next.There are two common types:*Normal edge*(`add_edge`) defines a fixed transition between nodes , *Conditional edge* (`add_conditional_edges`) uses a routing function to decide which node executes next based on the current state.

→ LANGGRAPH BUILDING BLOCKS

START

is a special virtual entry point in LangGraph. It represents the beginning of the workflow and determines which node executes first.

END

is a special virtual terminal point in LangGraph. It indicates that the workflow has finished executing. When the graph reaches END, execution stops and the final state is returned to the caller.

WHY IS THE STATE PASSED BETWEEN NODES?

The `state` allows nodes to share information throughout the workflow. Each node reads the current state, performs its task, and returns updates. LangGraph manages and merges these updates, enabling communication between nodes without global variables.

→ 4. IMPLEMENTATION AND RESULTS

4.1 FIRST WORKFLOW (PART II)

A two-field **TypedDict** state (**message**, **response**) and a single process node were wired as **START → process → END.** Invoking it with "Hello LangGraph!" returned **{'message': 'Hello LangGraph!', 'response': 'Agent received: Hello LangGraph!'},** matching the expected result and confirming the state and compile/invoke cycle.

4.2 LLM NODE AND TWO-STAGE AGENT (PARTS III AND IV)

The **chatbot** node calls the LLM and stores the reply in state. The connection test returned a greeting, and the three-sentence explanation of artificial intelligence was coherent and accurate. A second graph chained **analyzer → answer**: the analyzer identifies topic, intention and information needed, and the answer node uses that analysis to respond. For "**What is reinforcement learning?"** it produced a structured answer (definition, comparison with supervised and unsupervised learning, key components of an RL system). This shows how an intermediate reasoning step is carried through state.

→ 4. IMPLEMENTATION AND RESULTS

4.3 CONDITIONAL ROUTING (PARTS V AND VI)

A **classify_question** node labels each question as **mathematics, programming or artificial_intelligence**. The routing function **route_question** inspects the label and returns the next branch, with **ai** as the default. Three test questions were routed correctly

Question	Category	Result
What is the derivative of x^2?	mathematics	Correct answer, 2x
How do I create a REST API with Python?	programming	Complete FastAPI example with setup steps
What is reinforcement learning?	artificial_intelligence	University-level explanation

→ 4. IMPLEMENTATION AND RESULTS

4.4 TOOLS (PARTS VII AND VIII)

Two tools were defined with **@tool**: a calculator and **current_datetime**.
The lab's calculator uses **eval(),** which can execute arbitrary Python. It was kept only for comparison, and a safe version built on **ast** was used instead. It whitelists arithmetic operators **(+ - * / ** % //** and unary signs) and rejects everything else. Test results:

- **25 * 4 + 10** returned 110 with both versions;
- **__import__('os')** was rejected as **Invalid expression** by the safe version, which is the intended protection.

The tool-using graph binds the tools to the LLM, with a conditional edge **(tools_condition)** sending the flow to a ToolNode when the model emits a tool call, and an edge looping back to the agent. The traces confirm the expected behavior:

- "What is 125 * 38?" produced an AI message with a calculator call, a tool message with 4750, and a final answer of **4,750**;
- "What is the capital of Morocco?" produced no tool call and a direct answer, Rabat.

The model therefore decides when a tool is needed, rather than being forced to use one.

→ 4. IMPLEMENTATION AND RESULTS

4.5 MEMORY (PART IX)

The **add_messages** reducer appends messages instead of overwriting them. Persistence comes from compiling with a **MemorySaver** checkpointer and passing a **thread_id**.

Configuration	Behavior observed
No checkpointer	After "My name is Ahmed.", the question "What is my name?" got an answer saying the name was unknown
MemorySaver, same thread	Replied "Your name is Ahmed."
MemorySaver, different thread	Did not know the name, so threads are isolated

This confirms that memory in LangGraph is a property of the checkpointer and thread, not of the model.

4.6 VISUALIZATION

get_graph().draw_mermaid_png() was used to render the router graph, the tool-agent graph and the final assistant graph, with a fallback that prints the Mermaid source if PNG rendering is unavailable.

→ 5. CAPSTONE: INTELLIGENT UNIVERSITY ASSISTANT

5.1 ARCHITECTURE

The assistant is a LangGraph with this flow:

1. A classifier assigns the question to **COURSES, PROGRAMMING, MATHEMATICS, ADMINISTRATION or GENERAL,** or to **EMPTY / ERROR** for failure cases.
2. A conditional edge routes to the matching specialized agent, or to a fallback node for **EMPTY / ERROR.**
3. Each agent, built by a make_agent factory with its own system prompt, may call a tool. After the tool runs, control returns to the same agent, which writes the final answer.
4. The graph is compiled with a **MemorySaver** checkpointer so conversations persist per **thread_id.**

The requirements map onto the implementation as follows:

The requirements	Implementation
Question classifier	classify node
Specialized agents	One node per category via make_agent, each with a tailored system prompt
Memory	MemorySaver plus thread_id
Tools	calculator, current_datetime, plus Wikipedia as an extension
Conditional routing	route_after_classifier, route_after_agent, route_after_tools
Error handling	See below

→ 5. CAPSTONE: INTELLIGENT UNIVERSITY ASSISTANT

5.2 ERROR HANDLING

- Empty input: the classifier returns EMPTY and the fallback node asks the user to type a question.
- API failure: the classifier catches exceptions and returns ERROR; each agent catches exceptions and returns a polite message naming the exception type.
- Unknown label: any label outside the allowed set defaults to GENERAL.
- Invalid tool calls: ToolNode(..., handle_tool_errors=True) turns tool exceptions into messages the agent can recover from, instead of crashing the graph.
- Safety by prompt: the administration agent is told never to invent official rules or deadlines and to refer users to the administration office when unsure.

5.3 DEMONSTRATION RESULTS

The demonstration covers all categories, both built-in tools, the Wikipedia extension, memory and an error case

→ 5. CAPSTONE: INTELLIGENT UNIVERSITY ASSISTANT

Query	Category	Outcome
Topics in a machine learning course	COURSES	Overview of a typical curriculum, with an honest note that institution-specific syllabi are unavailable
Python function for factorial	PROGRAMMING	Commented function with explanation
What is 125 * 38 + 17?	MATHEMATICS	Calculator used; answer 4767
How to request an enrollment certificate	ADMINISTRATION	General guidance, stating that university-specific procedures are not known
What is today's date?	GENERAL	Date and time tool used; Tuesday, 3 November 2026
Empty input (whitespace)	EMPTY	Fallback message asking for a question
"My name is Ahmed." then "What is my name?" (same thread)	GENERAL	Name correctly recalled
Compute (15 + 27) * 3 using the calculator	MATHEMATICS	Calculator call returned 126, then a step-by-step explanation
Who was Rachid Yazami? Use Wikipedia.	GENERAL	Wikipedia tool used; summary of his work on lithium-ion graphite anodes and fluoride-ion batteries

→ 6. EVALUATION OF ROUTING BEHAVIOR

A small harness ran six labelled questions through the router and recorded the predicted category and total time.

Question	Expected	Got	Correct	Seconds
What is the derivative of x^2?	mathematics	mathematics	Yes	4.00
Solve 2x + 3 = 11	mathematics	mathematics	Yes	2.28
How do I reverse a list in Python?	programming	programming	Yes	10.16
Write a function to check for palindromes.	programming	programming	Yes	4.60
What is a convolutional neural network?	artificial_intelligence	artificial_intelligence	Yes	75.77
Explain overfitting.	artificial_intelligence	artificial_intelligence	Yes	27.53

Routing accuracy: 6/6 (100%).

→ 6. EVALUATION OF ROUTING BEHAVIOR

6.1 WHY 100% SHOULD NOT BE READ AS ROBUSTNESS

- **Small, unambiguous test set**. Two clear examples per category, with no mixed-topic or badly worded questions. "Write a Python function to compute the derivative of x^2" belongs to two categories and was not tested.
- **Lenient check.** The evaluation uses expected in got, a substring match that would accept "this is not mathematics". An exact comparison after normalization is stricter.
- **Silent fallback.** route_question also uses substring matching and sends anything unrecognized to the AI agent without an error or warning, so a wrong route would go unnoticed.
- **Only classification is measured.** Answer quality is not scored, and the timing covers the whole pipeline (classifier plus specialist), so it says little about routing cost. Latency varied widely, from 2.28 s to 75.77 s, which is consistent with a free model whose speed depends on provider load.

→ 6. EVALUATION OF ROUTING BEHAVIOR

WHY 100% SHOULD NOT BE READ AS ROBUSTNESS

- **Small, unambiguous test set**. Two clear examples per category, with no mixed-topic or badly worded questions. "Write a Python function to compute the derivative of x^2" belongs to two categories and was not tested.
- **Lenient check.** The evaluation uses expected in got, a substring match that would accept "this is not mathematics". An exact comparison after normalization is stricter.
- **Silent fallback.** route_question also uses substring matching and sends anything unrecognized to the AI agent without an error or warning, so a wrong route would go unnoticed.
- **Only classification is measured.** Answer quality is not scored, and the timing covers the whole pipeline (classifier plus specialist), so it says little about routing cost. Latency varied widely, from 2.28 s to 75.77 s, which is consistent with a free model whose speed depends on provider load.

8. CONCLUSION

The lab shows how LangGraph turns an LLM from a single call into a controllable workflow. State carries information between nodes, conditional edges let the model-driven classifier choose a path, tool nodes let the agent act and loop back, and checkpointers with thread identifiers provide conversational memory. The capstone assistant combines these elements with explicit error handling and a safe calculator, and it behaved correctly across all demonstration scenarios. The main weaknesses are in evaluation depth rather than in the graph design: the routing test is small and lenient, and the classifier and fallback logic would benefit from structured output and stricter validation.

APPENDIX: DELIVERABLES

- Executed notebook langgraph_lab.ipynb with outputs
- README.md (setup, how to run, design choices)
- Source code
- Architecture diagram exported from show_graph(assistant)
- Screenshots of execution
- This report